Lab 2 Assignment 2 Report

2025-12-02

```
knitr::opts_chunk$set(echo=TRUE, fig.width=6, fig.height=6, fig.align='center')
knitr::read_chunk("assignment2.R")
```

Assignment 2

Part 1

The data for the lab is imported and prepared as follows:

```
data <- read.csv2("./bank-full.csv")

# Exclude "duration" input variable
data <- data[ , -which(names(data) == "duration")]

# Convert all character columns to factors to avoid NAs during
#construction of decision trees
data[] <- lapply(data, function(x) if (is.character(x)) factor(x) else x)

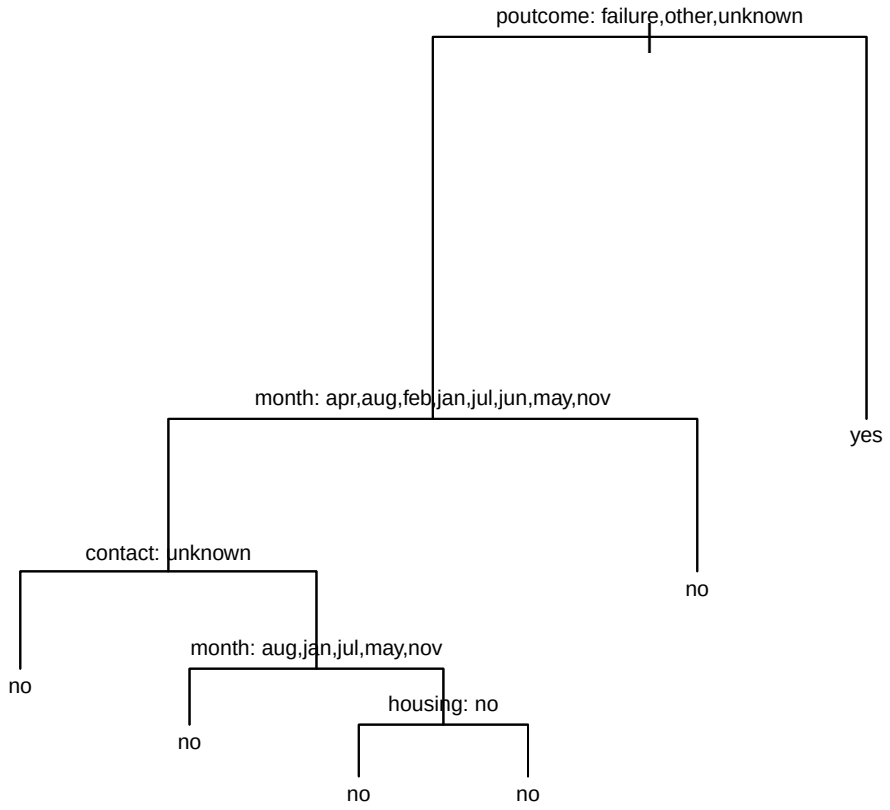
# Code taken from slide 20 of lecture 2a
n=dim(data)[1]
set.seed(12345)
id=sample(1:n, floor(n*0.4))
train=data[id,]
id1=setdiff(1:n, id)
set.seed(12345)
id2=sample(id1, floor(n*0.3))
valid=data[id2,]
id3=setdiff(id1,id2)
test=data[id3,]
```

Part 2 - Construction of three different decision trees

For the second part of the lab we fit three separate decision trees:

Decision tree with default settings

```
fit1 <- tree(y~., data=train)
plot(fit1)
text(fit1, pretty = 0, cex=0.7)
```



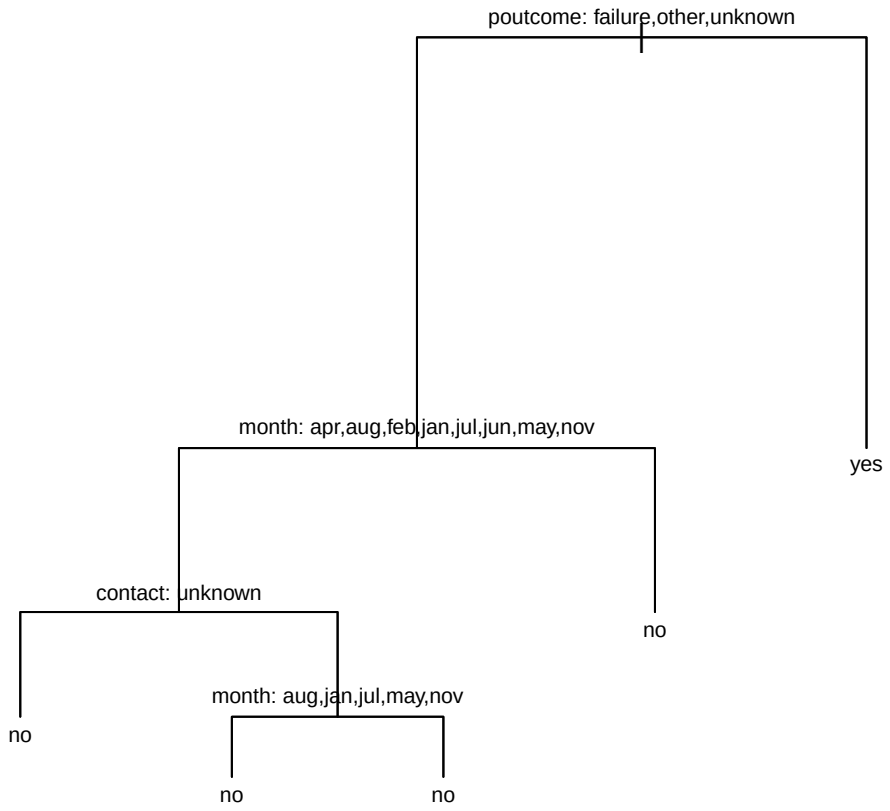
The default settings tree has the following misclassification rates for training and validation data.

```
## [1] "fit1 training misclassification:  0.104844061048441"
```

```
## [1] "fit1 validation misclassification:  0.109267861092679"
```

Decision tree with smallest allowed node size equal to 7000

```
ctrl <- tree.control(nobs <- nrow(train), minsize=7000)
fit2 <- tree(y~., data=train, control=ctrl)
plot(fit2)
text(fit2, pretty = 0, cex=0.7)
```



With a smallest allowed node size of 7000 the decision tree gets smaller (One less branch). This is because the smallest allowed node size controls how many observations are required in a leaf for a split to be allowed to happen. In the following table we can see the number of observations per leaf of the tree.

```
##
##    4    6    7    8    9
## 5130 9284 2414  640  616
```

Only "leaf 6" could potentially split into more branches (but does not due to other factors in the tree).

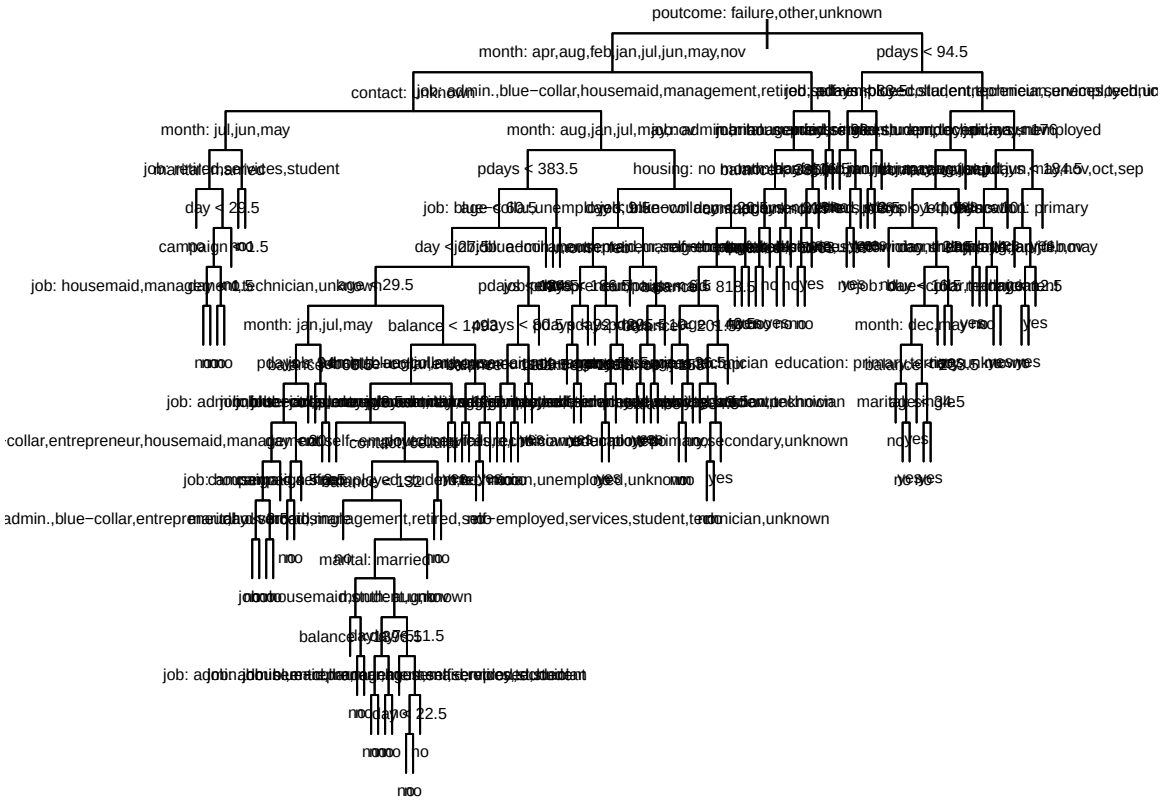
The minimum node size tree has the following misclassification rates for training and validation data.

```
## [1] "Fit2 training misclassification:  0.104844061048441"
```

```
## [1] "Fit2 validation misclassification:  0.109267861092679"
```

Decision tree with minimum deviance 0.0005

```
ctrl <- tree.control(nobs <- nrow(train), mindev=0.0005)
fit3 <- tree(y~., data=train, control=ctrl)
plot(fit3, type="u")
text(fit3, pretty = 0, cex=0.5)
```



With a minimum deviance of 0.0005 a split must achieve a 0.05% reduction in deviance in order to be allowed (compared to the 1% reduction required by the default settings). A minimum deviance of 0.0005 allows the

tree to grow much larger, as illustrated by the plot of our third tree, which shows a significant increase in size compared to previous trees.

The tree with minimum deviance 0.0005 has the following misclassification rates for training and validation data.

```
## [1] "Fit3 training misclassification:  0.0940057509400575"
```

```
## [1] "Fit3 validation misclassification:  0.111922141119221"
```

Best decision tree

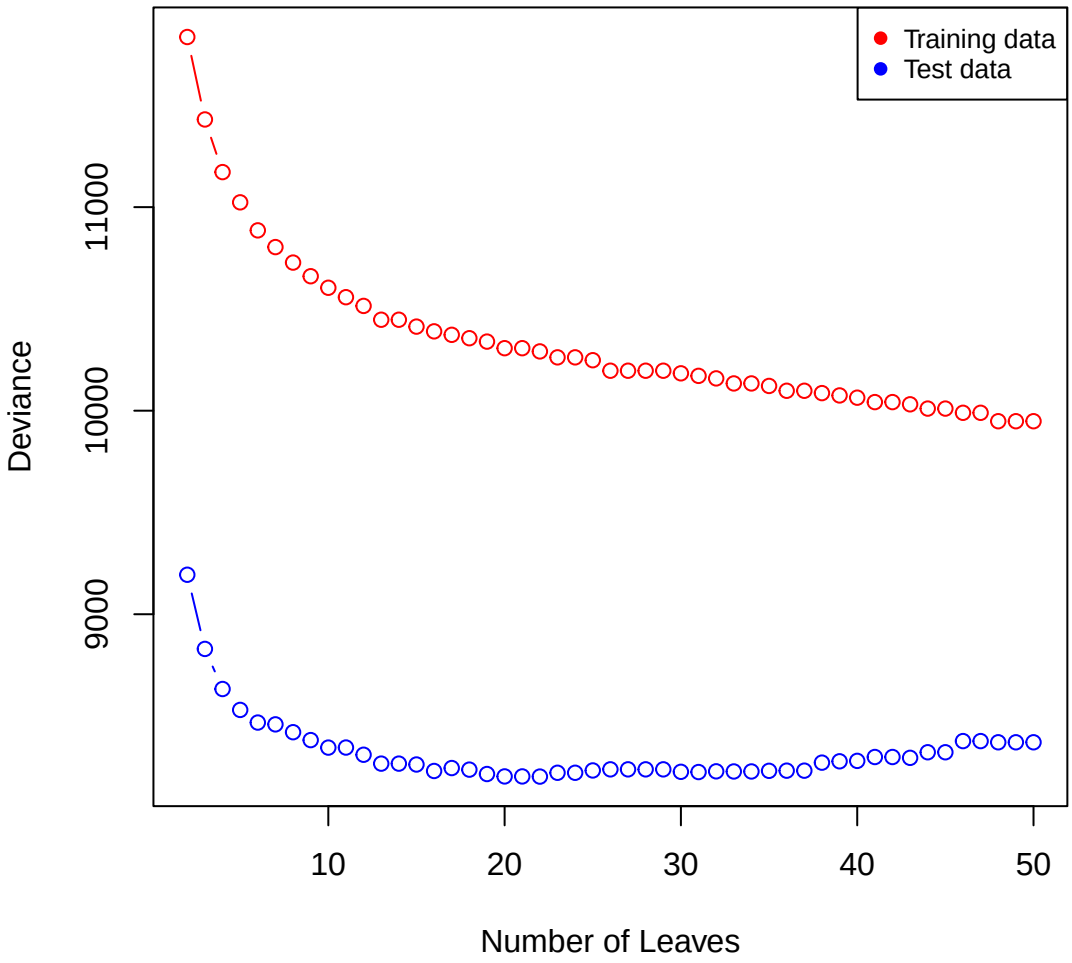
```
##                               DefaultTree MinSizeTree MinDevTree
## Training misclassification      0.1048441   0.1048441 0.09400575
## Validation misclassification   0.1092679   0.1092679 0.11192214
```

Looking at the training and validation data misclassification rates for our three decision trees we can see that the lowest training misclassification happens in the third tree with 9.4% misclassification. The same tree also has the highest validation misclassification which indicates overfitting.

The first and second decision trees have both identical training and validation misclassification. In this case a simpler/smaller tree is usually better as they are easier to interpret and are less likely to overfit to unseen data. For this reason the best decision tree is the minsize=7000 tree (the second tree).

Part 3 - Find optimal tree depth for min deviance 0.0005 tree

To find the optimal tree depth for our fit3 tree we prune the tree to all possible amounts of leaves between 2 and 50. For each tree we record its deviance for the training and validation data. This gives the following plot of the dependence of deviances for the training and validation data on the number of leaves.

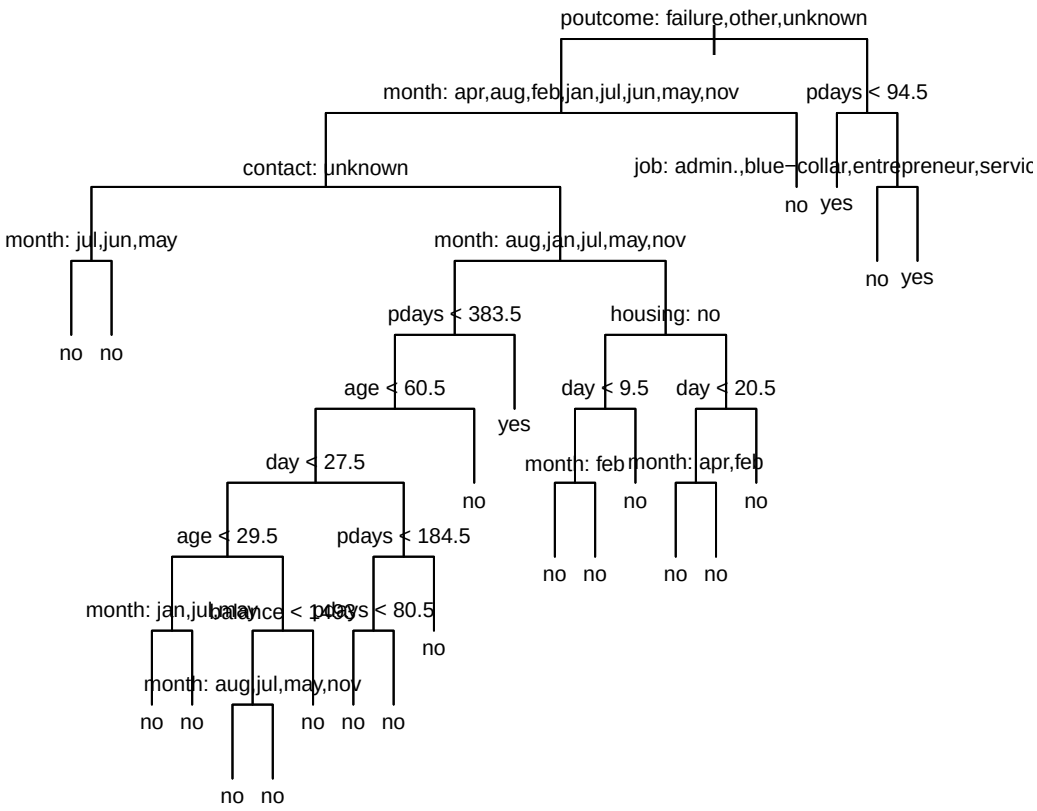


From the plot we see the deviance of the training data is always decreasing with more leaves. The deviance for the test data decreases until around 20 leaves whereafter it gets unstable. This is due to the decision tree overfitting to the training data at higher amounts of leaves. The optimal amount of leaves is gotten as follows:

```
optimal_leaves <- which.min(test_deviances[2:50]) + 1
print(paste("The optimal number of leaves is:", optimal_leaves))
```

```
## [1] "The optimal number of leaves is: 22"
```

With the optimal amount of leaves we create a optimal tree from fit3 with 22 leaves.



From the decision tree we can see that the most important variable for decision making is the outcome of previous campaigns towards a specific customer (poutcome).

A less expected variable that is shown to be quite important in the decision tree is the current month during a campaign. During the months of Mars, September, October and December we always predict "no" (depending on the outcome of first branch). Month continues to appear in branches further down the tree which indicates that the time of year during a campaign is relevant in all levels of prediction.

Another variable that shows up early and multiple times throughout the tree is "pdays" which represents the number of days since last attempted campaign towards a specific customer. In the right branch (from root node) we can see that no matter how many days its been since last attempted campaign we predict "yes" if the previous campaign succeeded.

In the left branch (from root node) the majority of outcomes result in "no". There is a single leaf resulting in "yes" when the contact is "known" and "pdays >= 383" since last contact. This could be interpreted as a new customers that is now receiving their first campaign call.

Part 4 - Statistics of optimal tree

```
##       Predicted
## Actual    no   yes
##    no  11872   107
##    yes  1371   214
```

From the confusion matrix of the optimal tree we calculate the accuracy and F1-score.

```
## [1] "Accuracy for optimal tree:  0.891035092892952"
```

This gives us a total misclassification of only about 10% which might appear good but in our data set the majority of outcomes result in "no" with few "yes". This means it might be more appropriate to use F1-score as a measure for predictive power to make sure the model is also good at predicted the less common outcomes.

```
## [1] "F1 Score for optimal tree:  0.224554039874082"
```

The F1-score is very low which indicates that the decision tree is not very good at predicting cases of "yes". For our tree the F1-score should be the preferred as the data set is unbalanced.

Part 5 - Decision tree classification with loss matrix

When applying the loss matrix given in the lab instructions to the predictions of the optimal tree we get the following confusion matrix:

```
##       Predicted
## Actual    no   yes
##    no  11030   949
##    yes   771   814
```

From this confusion matrix we calculate the accuracy and F1-score again.

```
## [1] "Accuracy for tree with loss matrix:  0.873193748156886"
```

```
## [1] "F1 Score for tree with loss matrix:  0.486260454002389"
```

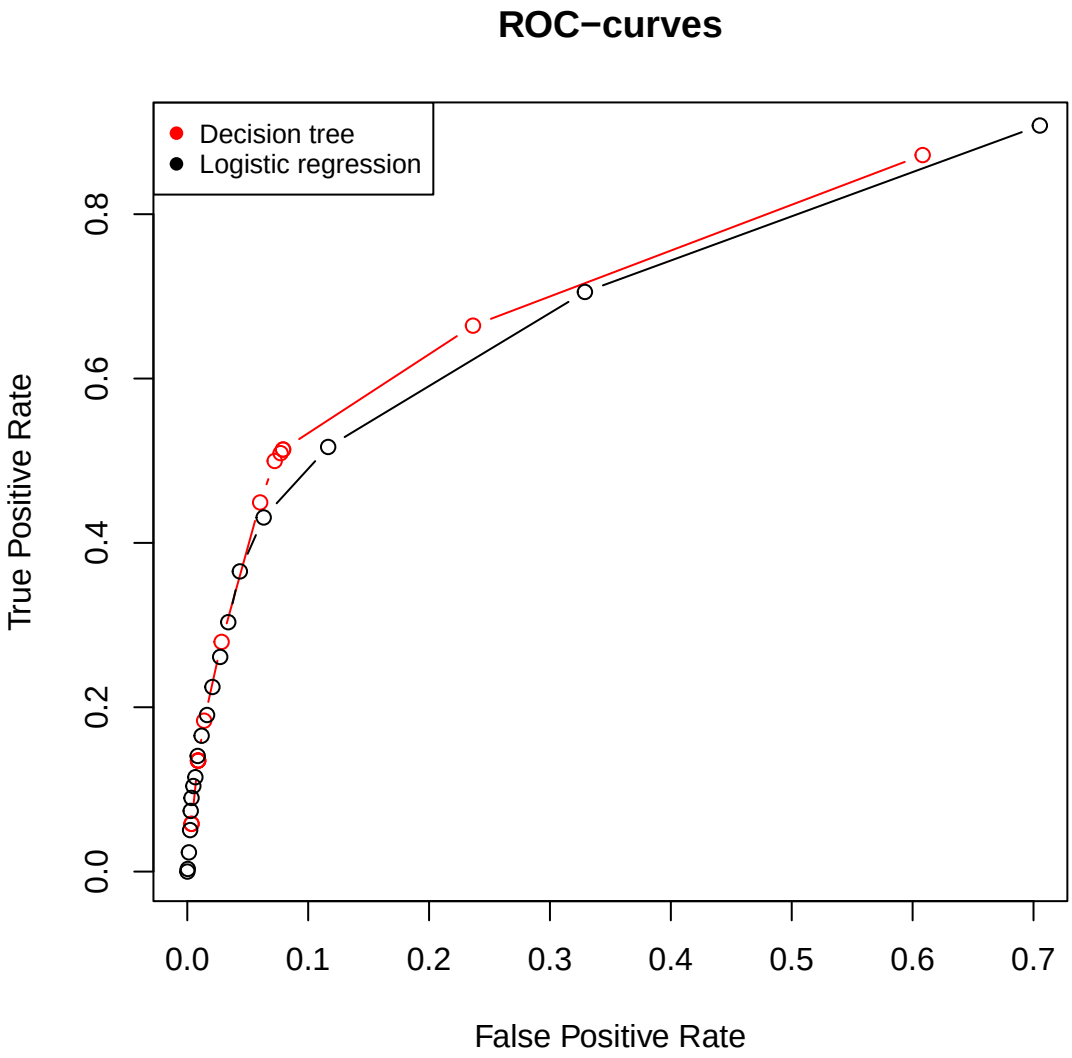
In the classification with the loss matrix the accuracy is slightly lower (87.3% compared to 89.2%) but the F1-score is substantially better (48.6% compared to 28.5%).

With the loss matrix the decision tree will try harder to predict "yes" correctly at the cost of slightly lower accuracy. Our data set is unbalanced which makes the F1-score a more important statistic to optimize for.

Part 6 - Compute TPR and FPR for a logistic regression model and decision tree

In the last part of the lab we compare the classification performance between the optimal tree and a logistic regression model using the following principle. $\hat{Y}$ = yes if $p(Y = \text{'yes'} \mid X) > \pi$, otherwise $\hat{Y}$ = no where π = 0.05,0.1,0.15, ... 0.9,0.95.

For each value of π we calculate the true positive rate (TPR) and false positive rate (FPR) for both models. Lastly all values are inserted into a plot.



Both the decision tree and logistic regression have very similar ROC-curves. The decision tree curve is after FPR=0.6 consistently very slightly above the logistic regression curve. From these plots the performance of the two models appear to be almost equal.

A precision-recall curve is better at showing performance on predictions on the positive outcomes of a model. As previously stated our data set is unbalanced with the majority of outcomes being “no”. This means a PR curve could be more appropriate, just as F1-score is more appropriate than accuracy for our decision trees.